

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Самостоятельная работа

Название «Информационная система студенческого общежития»

Выполнил

студент гр. 5130903/30003

Елакова В.О.

<подпись>

Руководитель

ст. преподаватель ВШПИ

Андрианова Е.Е.

<подпись>

«15» апреля 2025 г.

Санкт-Петербург
2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	4
Описание предметной области.....	4
Требования к базе данных.....	9
ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....	11
Концептуальная модель	11
Логическая модель.....	12
Физическая модель	28
РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ.....	29
Тестирование ограничений.....	29
Обратное проектирование.....	31
РАЗРАБОТКА ЗАПРОСОВ	32
ЗАКЛЮЧЕНИЕ	36
СПИСОК ЛИТЕРАТУРЫ.....	37
ПРИЛОЖЕНИЕ	38
Приложение 1	38
Приложение 2	55
Приложение 3	61
Приложение 4	63

ВВЕДЕНИЕ

В современном мире автоматизация играет ключевую роль в оптимизации различных процессов. Одним из важных аспектов эффективного управления общежитием является организация и структурирование информации о проживающих, оплатах за проживание, корпусах и других существенных данных. Неоптимизированное хранение информации приводит к ошибкам, задержкам и неразумному потреблению ресурсов. В связи с этим данная тема является актуальной.

Целью данной работы является проектирование и создание реляционной базы данных, обеспечивающей эффективное управление информацией об общежитии.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Проанализировать предметную область
 - 1.1. Определить бизнес-правила.
 - 1.2. Выявить ключевые сущности и их взаимосвязи.
2. Разработать концептуальную, логическую и физическую модели.
3. Провести нормализацию отношений.
4. Сделать представление базы данных.
5. Проверить корректность работы базы данных.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Описание предметной области

Объекты предметной области и их информационные потребности представлены в таблице 1.

Пользователь системы	Бизнес-цели
Корпус	Хранит данные об адресе, количестве комнат, числе проживающих, категории жильцов, дежурном.
Проживающий	• Проживает в определенном корпусе общежития. • Предоставляет необходимую личную информацию.
История платежей	В нее фиксируются: 1. Дата платежа 2. Сумма 3. Серия и номер проживающего 4. ФИО проживающего
Архив	Хранит информацию о выселенных проживающих.
Нарушение	Хранит информацию о нарушителе, дежурном, дате происшествия, виде нарушения.
Дежурный	Работает в общежитие. Предоставляет необходимую личную информацию.
График дежурств	Хранит информацию о дежурных, их рабочих днях, адресе рабочего места.

Проверка	Хранит информацию о дате проведения проверки, корпусе, в котором она проводилась, комнате, которую проверяли и рейтинг, выставленном в результате проверки.
История переселений	В нее вносятся данные о: • Проживающий • Комната • Дата • Корпус
Родитель	Предоставляет необходимую о себе информацию.
Комната	Имеет номер и информацию о корпусе, в котором находится.
История академических изменений	Хранит данные о поступлении, факультете, специальности, академическом отпуске.

Таблица 1 Объекты предметной области

Свойства объектов

Корпус:

- Адрес корпуса
- Количество комнат
- Число проживающих
- Количества жильцов по категориям
- Категории жильцов
- Дежурный

История академических изменений

- Курс
- Специальность
- Группа

- Факультет
- Форма обучения
- Дата начала
- Дата конца

Проживающий:

- Категория проживающего
- ФИО
- Паспортные данные
- Дата рождения
- Место рождения
- Место прописки
- Тариф
- Номер телефона
- Дата заселения

Подтипы проживающего:

Студент:

- ФИО студента
- Паспортные данные
- Родитель

Молодой преподаватель:

- ФИО преподавателя
- Паспортные данные
- Название кафедры

Аспирант:

- ФИО аспиранта
- Паспортные данные
- Название кафедры
- Форма обучения

- Специальность

История платежей:

- Дата платежа
- Сумма платежа
- Паспортные данные оплатившего
- ФИО оплатившего

Архив:

- Дата выселения
- ФИО выселенного
- Паспортные данные
- Категория проживающего (молодой преподаватель, студент, аспирант)

Нарушение:

- ФИО нарушителя
- Дата
- Дежурный
- Вид нарушения

Дежурный:

- ФИО дежурного
- Паспорт
- Номер телефона
- Дата рождения
- Место прописки

График дежурств:

- ФИО дежурного
- Паспортные данные дежурного
- Адрес рабочего места
- Дата

Проверка:

- Номер комнаты

- Дата
- Рейтинг
- Дежурный

История переселений:

- Комната
- Дата выселения
- Дата заселения
- ФИО проживающего
- Паспортные данные

Родитель:

- ФИО
- Номер телефона
- Паспортные данные

Комната:

- Номер комнаты
- Адрес корпуса

Связи между объектами.

Дежурный имеет один График дежурств. В Графике дежурств указано один, много Дежурных.

Нарушение может быть совершено определенным Проживающим. Проживающий может совершить ноль, одно, много Нарушений.

Нарушение может зафиксировать только Дежурный. Дежурный фиксирует ноль, одно, много Нарушений.

У каждого Проживающего есть ноль, одна, много записей в Истории платежей. Запись в Истории платежей соответствует одному Проживающему.

Информация о Родителе хранится только для Студента. У каждого Студента есть ноль, один, два Родителя.

В Истории переселений хранится ноль, одна, много записей о Проживающих. Проживающий может переселяться ноль, один, много раз, что фиксируется в Истории переселений.

В Графике дежурств записаны Корпуса, в которых работают Дежурные. Корпусу соответствует один Дежурный в Графике дежурств.

История переселений хранит одну, много записей о переселениях в Комнату. Одной записи в Истории переселений соответствует одна Комната.

Проверка содержит одна, много записей о Комнате. Одной записи в Проверке соответствует одна Комната.

Проверку проводит только один Дежурный. Дежурный проводит ноль, одну, много Проверок.

Корпус имеет ноль, одну, много Комнат. Комната относится к одному Корпусу.

Одной записи в Архиве соответствует один Проживающий. Одному Проживающему соответствует ноль или одна запись в Архиве.

Проживающий может быть либо Студентом, либо Аспирантом, либо Молодым преподавателем. Студент, Аспирант, Молодой преподаватель являются Проживающими.

У Студента может быть одно, много изменений в Истории академических изменений. Одной записи в Истории академических изменений соответствует один Студент.

Требования к базе данных

Информационная система должна хранить данные, обрабатывать их. Информационная система должна иметь возможность масштабироваться, то есть, например, можно добавлять новые корпуса, комнаты, проживающих.

В общежитии могут проживать студенты, аспиранты, молодые преподаватели до 35 лет. Преподавателям предоставляются отдельные комнаты. Студенты и аспиранты разделяются на контрактных и бюджетных. Для студентов и аспирантов установлены разные тарифы проживания в зависимости

от формы обучения. В комнате может проживать до трех человек. Аспиранты и студенты не могут жить вместе в комнате.

База данных должна хранить полную информацию о проживающем: ФИО, паспортные данные, место прописки и т.п. Система должна хранить данные об оплате проживания. Информация о выселенных проживающих хранится в архиве.

Если студент нарушает регламент общежития, то сведения об этом заносятся в базу. Отмечается вид нарушения, дата, дежурный. О дежурном хранится информация в базе данных, как о сотруднике. У дежурных есть график дежурств. У каждого корпуса свой дежурный. Дежурные могут выходить в разные корпуса.

Также в общежитии проводятся проверки комнат, по результатам отмечаются корпус и дата проведения проверки, а также рейтинг.

Необходимо ввести учет академических изменений, например, студент может переводиться с одного направления на другое.

В представлении должно быть указано: список проживающих в общежитии людей, указание их категории, стоимость проживания.

ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

Концептуальная модель

Для построения инфологической модели необходимы список сущностей, список их атрибутов, выявление взаимосвязей между сущностями и определение пользователей. Первые три пункта были определены в прошлом разделе.

Пользователи системы коменданты, кураторы, деканат, отдел кадров.

Сущности Родитель и Студент имеют отношение многие-ко-многим, поэтому требуется добавить отношение Родственник. Один студент может иметь ноль, один, много Родственников. Один Родитель может иметь несколько Родственников, проживающих в общежитии.

После преобразований модель будет выглядеть следующим образом (см. рис. 1).

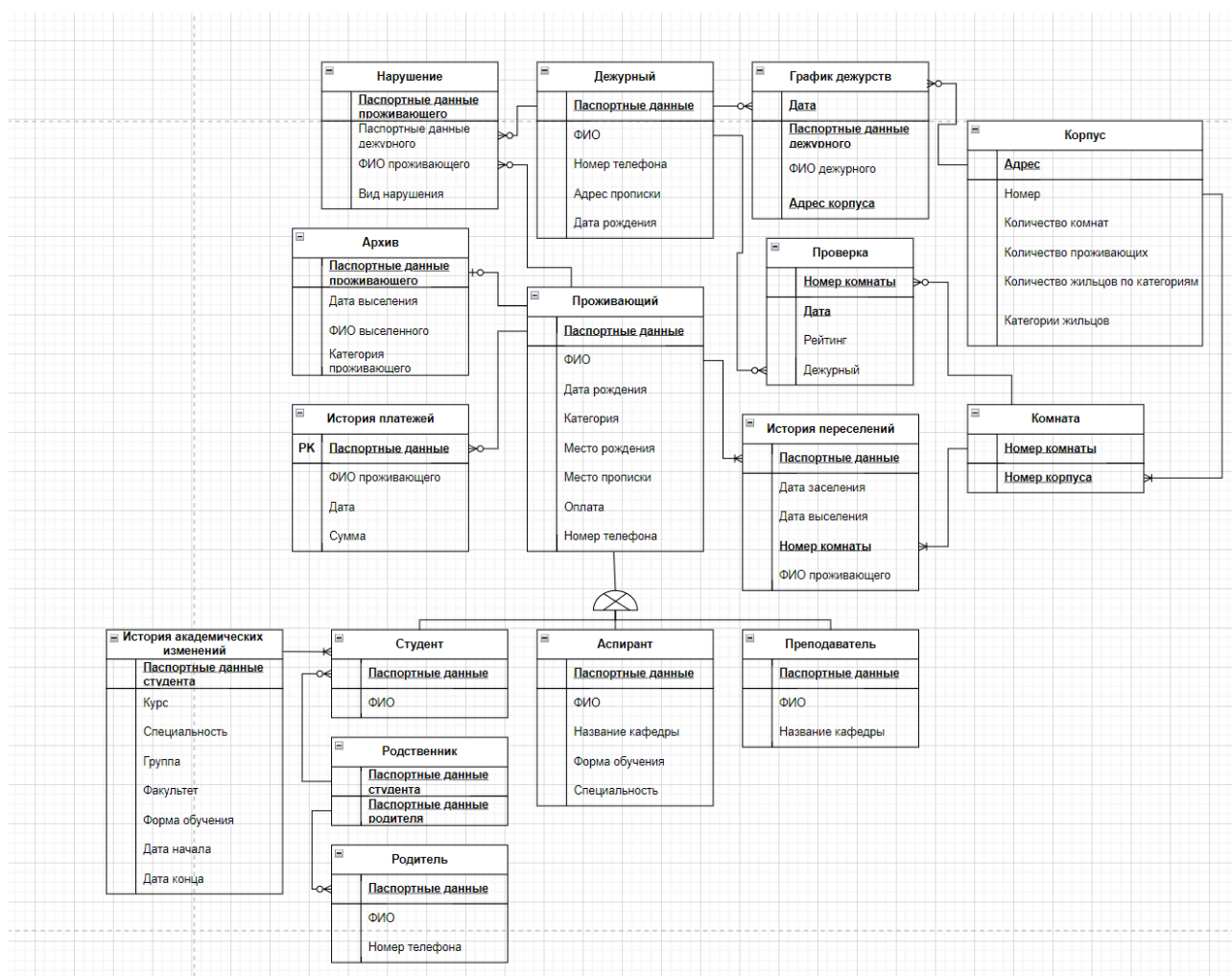


Рис. 1 Концептуальная модель

Логическая модель

Требуется провести нормализацию отношений. Первым будет рассмотрено отношение «Дежурный».

Дежурный	
РК	Паспортные данные
	Номер телефона
	Адрес прописки
AK1.4	Дата рождения
AK1.1	Фамилия
AK1.2	Имя
AK1.3	Отчество

Рис. 2 Отношение "Дежурный"

Потенциальные ключи: {Паспортные данные}, {Фамилия, Имя, Отчество, Дата рождения}

Первичный ключ: {Паспортные данные}

Функциональные зависимости: {Паспортные данные} → {Адрес прописки}

Неключевые атрибуты: Номер телефона, Адрес прописки

Отношение «Дежурный» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Дежурный» находится в 1НФ, и каждый его неключевой атрибут функционально полно зависит от первичного ключа, значит оно находится в 2НФ.

Отношение «Дежурный» находится в 3НФ, так как находится в 2НФ и не содержит транзитивных функциональных зависимостей между неключевыми атрибутами и каким-либо потенциальным ключом.

Отношение «Дежурный» находится в НФБК, так как нет пересекающихся потенциальных ключей.

Отношение «Дежурный» находится в 4НФ и 5НФ, так как содержит неключевые атрибуты.

График дежурств		
PK1.1	Дата	
PK1.2	Паспортные данные дежурного	
PK1.3	ID_корпуса	

Рис. 3 Отношение "График дежурств"

Потенциальные ключи: {Дата, Паспортные данные, ID_корпуса}

Первичный ключ: {Дата, Паспортные данные, ID_корпуса }

Неключевых атрибутов нет, функциональных зависимостей тоже нет.

Отношение «График дежурств» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «График дежурств» находится в 1НФ, неключевых атрибутов нет, значит оно находится в 2НФ.

Отношение «График дежурств» находится в 3НФ, так как нет неключевых атрибутов.

Отношение «График дежурств» находится в НФБК, так как есть только один потенциальный ключ.

В отношении $R(X,Y,Z)$ существует многозначная зависимость $X \twoheadrightarrow Y$ тогда и только тогда, когда множество значений атрибута Y , соответствующее паре значений атрибутов X и Z , зависит только от X и не зависит от Z .

Представим, что существует многозначная зависимость Дата->->Паспортные данные, тогда множество значений атрибута Паспортные данные зависит только от Дата и не зависит от ID_корпуса, но Паспортные данные не зависят от Даты дежурства. Многозначная зависимость отсутствует.

Представим, что существует многозначная зависимость Паспортные данные->->Дата, тогда множество значений атрибута Дата зависит только от Паспортные данные и не зависит от ID_корпуса, но Дата не зависит от Паспортных данных. Многозначная зависимость отсутствует.

Представим, что существует многозначная зависимость Дата->->ID_корпуса, тогда множество значений ID_корпуса зависит только от Дата и не зависит от Паспортные данные, но ID_корпуса не зависит от Даты дежурства. Многозначная зависимость отсутствует.

Представим, что существует многозначная зависимость Паспортные данные->->ID_корпуса, тогда множество значений атрибута ID_корпуса зависит только от Паспортные данные и не зависит от Дата, но ID_корпуса не зависит от Паспортных данных. Многозначная зависимость отсутствует.

Представим, что существует многозначная зависимость ID_корпуса ->->Дата, тогда множество значений атрибута Дата зависит только от ID_корпуса и не зависит от Паспортные данные, но Дата не зависит от ID_корпуса. Многозначная зависимость отсутствует.

Представим, что существует многозначная зависимость ID_корпуса ->->Паспортные данные, тогда множество значений атрибута Паспортные данные зависит только от ID_корпуса и не зависит от Дата, но Паспортные данные не зависят от ID_корпуса. Многозначная зависимость отсутствует.

Отношение «График дежурств» находится в 4НФ, так как нет многозначных зависимостей.

Определение: В отношении $R(X, Y, Z)$ существует зависимость соединения $*(X, Y, Z)$ тогда и только тогда, когда декомпозиция отношения R на $R_1(X, Y)$, $R_2(Y, Z)$ и $R_3(X, Z)$ происходит без потерь.

В нашем случае если мы разделим отношение «График дежурств» на три новых произойдут потери, например, если будет отношение, состоящее из Дата и ID_корпуса, то неизвестно кто дежурный. Значит отношение находится в 5НФ.

Корпус	
	ID_корпуса
	Количество комнат
	Количество проживающих
	Количество студентов
	Количество аспирантов
	Количество молодых преподавателей
	Адрес

Рис. 4 Отношение "Корпус"

Потенциальные ключи: {Адрес}, {ID_корпуса}

Первичный ключ: {ID_корпуса}

Неключевые атрибуты: Количество комнат, Количество проживающих, Количество студентов, Количество аспирантов, Количество молодых преподавателей

Функциональные зависимости: {ID_корпуса} → Количество комнат, Количество проживающих, Количество студентов, Количество аспирантов, Количество молодых преподавателей

{Количество студентов, Количество аспирантов, Количество молодых преподавателей} → Количество проживающих

Отношение «Корпус» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Корпус» находится в 1НФ, и каждый его неключевой атрибут функционально полно зависит от первичного ключа, значит оно находится в 2НФ.

Для приведения к 3НФ следовало бы создать новое отношение, однако атрибут Количество проживающих вычисляем, поэтому его проще просто удалить.

Корпус	
	<u>ID_корпуса</u>
	Количество комнат
	Количество студентов
	Количество аспирантов
	Количество молодых преподавателей
	Адрес

Рис. 5 Обновленное отношение "Корпус"

Потенциальные ключи: {Адрес}, {ID_корпуса}

Первичный ключ: {ID_корпуса}

Неключевые атрибуты: Количество комнат, Количество студентов, Количество аспирантов, Количество молодых преподавателей

Функциональные зависимости: {ID_корпуса} → Количество комнат, Количество студентов, Количество аспирантов, Количество молодых преподавателей

Отношение «Корпус» находится в 3НФ, так как находится в 2НФ и не содержит транзитивных функциональных зависимостей между неключевыми атрибутами и каким-либо потенциальным ключом.

Отношение «Корпус» находится в НФБК, так как нет пересекающихся потенциальных ключей.

Отношение «Корпус» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

Комната	
	<u>ID_комнаты</u>
	<u>ID_корпуса</u>

Потенциальный ключ: {ID_корпуса, ID_комнаты}

Первичный ключ: {ID_корпуса, ID_комнаты}

Неключевых атрибутов нет, функциональных зависимостей нет.

Отношение «Комната» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Комната» находится в 2НФ, так как его неключевые атрибуты функционально полно зависят от первичного ключа.

Отношение «Комната» находится в 3НФ, так как нет неключевых атрибутов.

Отношение «Комната» находится в НФБК, так как пересекающиеся ключи отсутствуют.

Отношение «Комната» находится в 4НФ и 5НФ, так как ключевых атрибутов менее 3-х.

Проверка	
РК	<u>ID_комнаты</u>
РК	<u>Дата</u>
	Рейтинг
	Дежурный

Рис. 7 Отношение "Проверка"

Потенциальные ключи: {Дата, ID_комнаты}

Первичный ключ: {Дата, ID_комнаты}

Неключевые атрибуты: Рейтинг, Дежурный

Функциональные зависимости: {ID_комнаты, Дата} → Рейтинг, Дежурный

Отношение «Проверка» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат

атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Проверка» находится в 2НФ, так как его неключевые атрибуты функционально полно зависят от первичного ключа.

Отношение «Проверка» находится в 3НФ, так как не содержит транзитивных функциональных зависимостей между неключевыми атрибутами и каким-либо потенциальным ключом.

Отношение «Проверка» находится в НФБК, так как пересекающиеся ключи отсутствуют.

Отношение «Проверка» находится в 4НФ и 5НФ, так как ключевых атрибутов менее 3-х и имеются неключевые атрибуты.

Проживающий	
РК	<u>Паспортные данные</u>
	Дата рождения
	Категория
	Место рождения
	Место прописки
	Оплата
	Номер телефона
	Фамилия
	Имя
	Отчество

Рис. 8 Отношение "Проживающий"

Потенциальные ключи: {Дата рождения, Фамилия, Имя, Отчество},
{Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевые атрибуты: Категория, Место рождения, Место прописки, Оплата, Номер телефона

Функциональные зависимости: {Паспортные данные} → {Место прописки}

Отношение «Проживающий» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Проживающий» находится в 2НФ, так как все его неключевые атрибуты зависят функционально полно от первичного ключа.

Отношение «Проживающий» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Проживающий» находится в НФБК, так как нет пересекающихся потенциальных ключей.

Отношение «Проживающий» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.



Рис. 9 Отношение "Студент"

Потенциальный ключ: {Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевых атрибутов нет, функциональных зависимостей нет.

Отношение «Студент» находится в 1НФ, все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Студент» находится в 2НФ и 3НФ, так как нет неключевых атрибутов.

Отношение «Студент» находится в НФБК, так как нет пересекающихся потенциальных ключей.

Отношение «Студент» находится в 4НФ и 5НФ, так как ключевых атрибутов менее 3-х.

Аспирант	
РК	<u>Паспортные данные</u>
	Название кафедры
	Форма обучения
	Специальность

Рис. 10 Отношение "Аспирант"

Потенциальные ключи: {Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевые атрибуты: Название кафедры, Форма обучения, Специальность.

Функциональные зависимости: {Паспортные данные} -> Название кафедры, Форма обучения, Специальность.

Отношение «Аспирант» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Аспирант» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «Аспирант» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Аспирант» находится в НФБК, так как нет пересекающихся ключей.

Отношение находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

Преподаватель	
РК	<u>Паспортные данные</u>
	Название кафедры

Рис. 11 Отношение "Преподаватель"

Потенциальные ключи: {Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевые атрибуты: Название кафедры

Функциональные зависимости: {Паспортные данные} → Название кафедры

Отношение «Преподаватель» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Преподаватель» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «Преподаватель» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Преподаватель» находится в НФБК, так как нет пересекающихся ключей.

Отношение «Преподаватель» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

Родитель	
РК	<u>Паспортные данные</u>
	Фамилия
	Имя
	Отчество
	Номер телефона

Рис. 12 Отношение "Родитель"

Потенциальные ключи: {Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевые атрибуты: Фамилия, Имя, Отчество, Номер телефона

Отношение «Родитель» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Родитель» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «Родитель» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Родитель» находится в НФБК, так как нет пересекающихся ключей.

Отношение «Родитель» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

Родственник	
РК	<u>Паспортные данные студента</u>
РК	<u>Паспортные данные родителя</u>

Рис. 13 Отношение "Родственник"

Потенциальные ключи: {Паспортные данные студента, Паспортные данные родителя}

Первичный ключ: {Паспортные данные студента, Паспортные данные родителя}

Неключевые атрибуты и функциональные зависимости отсутствуют.

Отношение «Родственник» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Родственник» находится в 2НФ, так как неключевых атрибутов нет.

Отношение «Родственник» находится в 3НФ, так как неключевых атрибутов нет.

Отношение «Родственник» находится в НФБК, так как нет пересекающихся ключей.

Отношение «Родственник» находится в 4НФ и 5НФ, так как ключевых атрибутов менее 3-х.

История академических изменений	
РК	Паспортные данные студента
	Курс
	Специальность
	Группа
	Факультет
	Форма обучения
РК	Дата начала
	Дата конца

Рис. 14 Отношение "История академических изменений"

Потенциальные ключи: {Паспортные данные, Дата начала}

Первичный ключ: {Паспортные данные, Дата начала}

Неключевые атрибуты: Курс, Специальность, Группа, Факультет, Форма обучения, Дата конца

Функциональные зависимости: {Паспортные данные, Дата начала}-> Курс, Специальность, Группа, Факультет, Форма обучения, Дата конца

Отношение «История академических изменений» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «История академических изменений» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «История академических изменений» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «История академических изменений» находится в НФБК, так как нет пересекающихся ключей.

Отношение «История академических изменений» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

История переселений	
	<u>Паспортные данные</u>
	Дата заселения
	Дата выселения
	<u>ID_комнаты</u>

Рис. 15 Отношение "История переселений"

Потенциальные ключи: {Паспортные данные, ID_комнаты}

Первичный ключ: {Паспортные данные, ID_комнаты}

Неключевые атрибуты: Дата заселения, Дата выселения

Функциональные зависимости: {Паспортные данные, ID_комнаты}-> Дата заселения, Дата выселения

Отношение «История переселений» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «История переселений» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «История переселений» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «История переселений» находится в НФБК, так как нет пересекающихся ключей.

Отношение «История переселений» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

История платежей	
РК	<u>Паспортные данные</u>
РК	<u>Дата</u>
	Сумма

Рис. 16 Отношение "История платежей"

Потенциальные ключи: {Паспортные данные, Дата}

Первичный ключ: {Паспортные данные, Дата}

Неключевые атрибуты: Сумма

Функциональные зависимости: {Паспортные данные, Дата}-> Сумма

Отношение «История платежей» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «История платежей» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «История платежей» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «История платежей» находится в НФБК, так как нет пересекающихся ключей.

Отношение «История платежей» находится в 4НФ и 5НФ, так как есть неключевой атрибут.

Архив	
	Паспортные данные проживающего
	Дата выселения
	Фамилия
	Имя
	Отчество
	Категория проживающего

Рис. 17 Отношение "Архив"

Потенциальные ключи: {Паспортные данные}

Первичный ключ: {Паспортные данные}

Неключевые атрибуты: Дата выселения, Фамилия, Имя, Отчество, Категория проживающего

Отношение «Архив» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Архив» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «Архив» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Архив» находится в НФБК, так как нет пересекающихся ключей.

Отношение «Архив» находится в 4НФ и 5НФ, так как есть неключевые атрибуты.

Нарушение	
РК	Паспортные данные проживающего
РК	Паспортные данные дежурного
	Вид нарушения
РК	Дата

Рис. 18 Отношение "Нарушение"

Потенциальные ключи: {Паспортные данные проживающего, Паспортные данные дежурного, Дата}

Первичный ключ: {Паспортные данные проживающего, Паспортные данные дежурного, Дата}

Неключевые атрибуты: Вид нарушения

Отношение «Нарушение» находится в 1НФ, так как все столбцы имеют уникальные имена, определён его первичный ключ, все атрибуты отношения содержат атомарные значения, нет упорядочивания строк сверху-вниз, нет упорядочивания столбцов слева-направо.

Отношение «Нарушение» находится в 2НФ, так как его неключевые атрибуты находятся в функционально полной зависимости.

Отношение «Нарушение» находится в 3НФ, так как нет функциональных зависимостей между неключевыми атрибутами.

Отношение «Нарушение» находится в НФБК, так как нет пересекающихся ключей.

Отношение «Нарушение» находится в 4НФ и 5НФ, так как есть неключевой атрибут.

Логическая модель представлена на рис. 19.

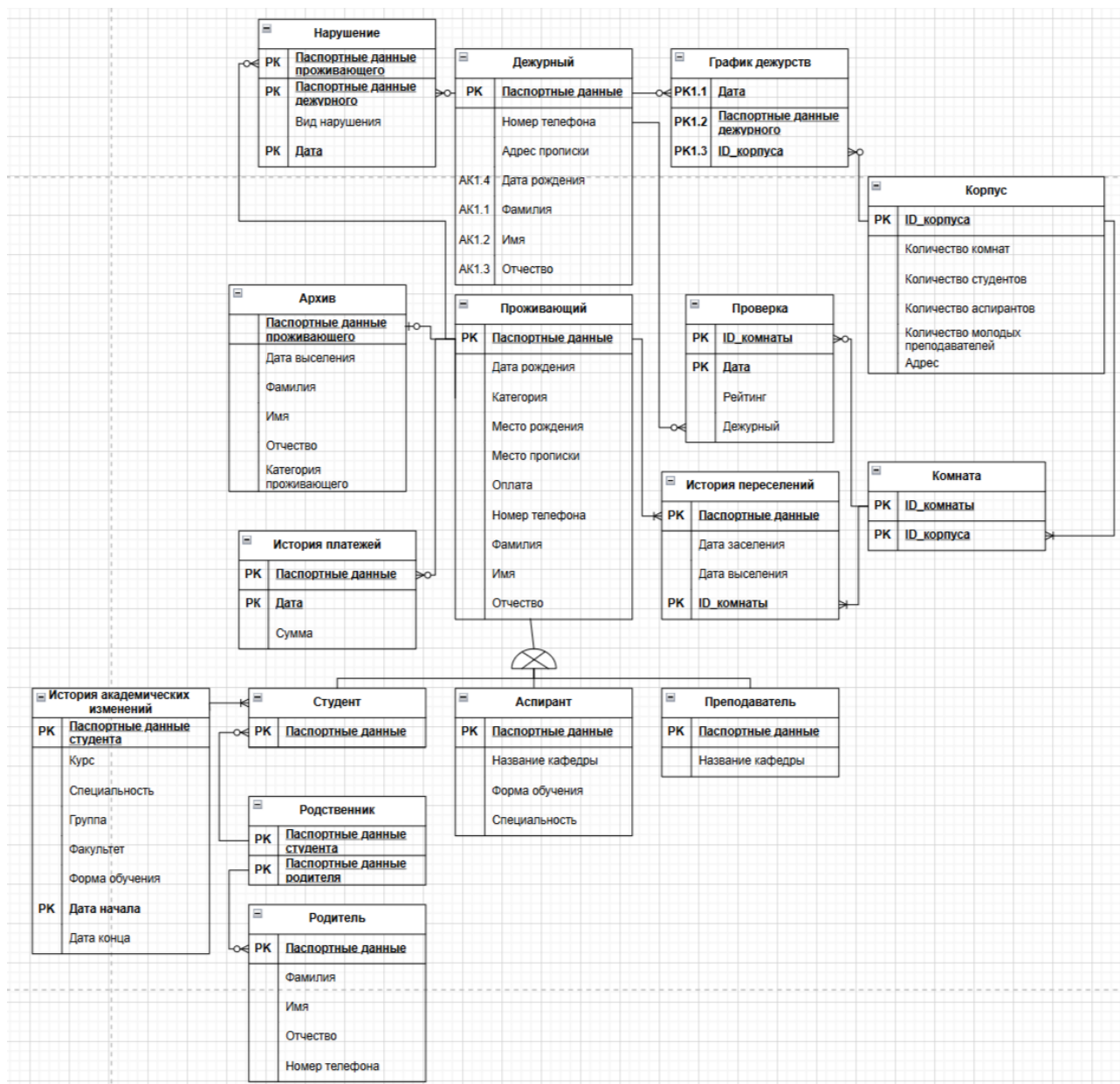


Рис. 19 Логическая модель

Физическая модель

Требуется определить типы данных для атрибутов всех сущностей. Для любых паспортных данных – VARCHAR(20), для дат – DATE, кроме даты нарушения – TIMESTAMP, для количества чего-либо, сумма платежа, рейтинг, курс – INTEGER, сумма платежа, оплата – DECIMAL(10, 2), ID – SERIAL, остальные атрибуты будут иметь тип данных VARCHAR различной длины.

РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ

Тестирование ограничений

В приложении «Приложение 1» представлен код реализации базы данных в «Приложение 2» ее заполнение. Далее внимание будет уделено важным ограничениям в текущей системе, код для проверки работоспособности представлен в «Приложение 3». В ходе реализации были установлены такие ограничения СНЕСК:

- Ограничение на возраст преподавателя.
- Не отрицательность суммы платежа, оплаты, количества проживающих по категориям и комнат.
- Ограничение на ввод номера телефона, паспортных данных, определенный список значений для категории проживающего и формы обучения.
- Соответствие действительности дат начала и конца событий.

Проверим корректность работы ограничения на возраст. Для этого попробуем вставить человека с датой рождения 01.01.1983.

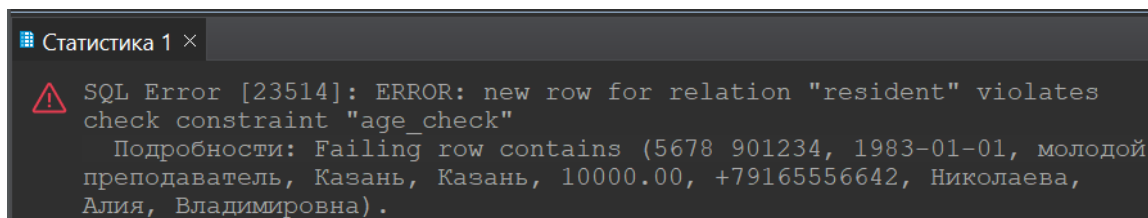


Рис. 20 Проверка работоспособности ограничения на возраст молодого преподавателя

Как видно из рис. 20 ограничение справилось успешно.

Далее проверим ограничение на не отрицательность суммы платежа. Рис. 21 отражает работоспособность данного СНЕСК. Остальные ограничения на не отрицательность работают по подобию рассмотренного примера.

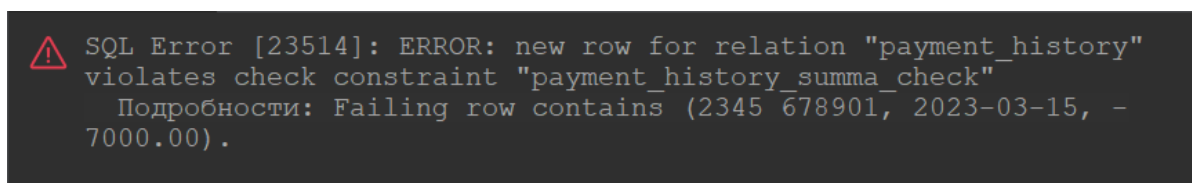


Рис. 21 Проверка ограничения на не отрицательность суммы платежа

Протестируем выполнение CHECK для атрибута «Категория» сущности «Проживающий». Ограничение заключается в том, что есть только определенные значения для категорий: студент, аспирант, молодой преподаватель. Из рис. 22 видно, что ограничение работает исправно. Аналогично работает CHECK для формы обучения: бюджет/ контракт.

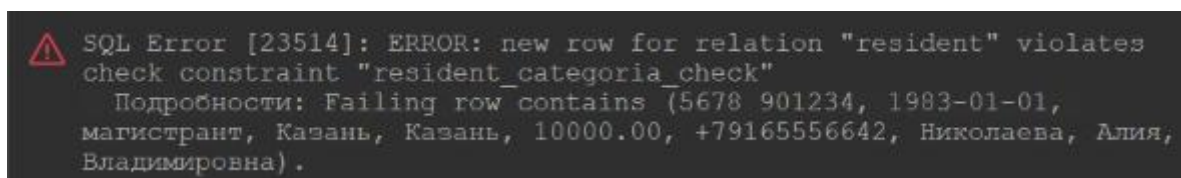


Рис. 22 Проверка ограничения на категории проживающего

Выполним тестирование условия на соответствие шаблону номера телефона. На рис. 23 отображен положительный результат работоспособности ограничения. По аналогии работает ограничение для паспортных данных.

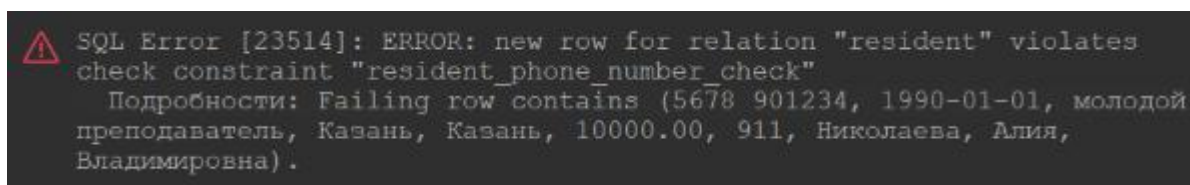


Рис. 23 Проверка ограничения для номера телефона

Последним ограничением CHECK является проверка на соответствие действительности дат начала и конца.

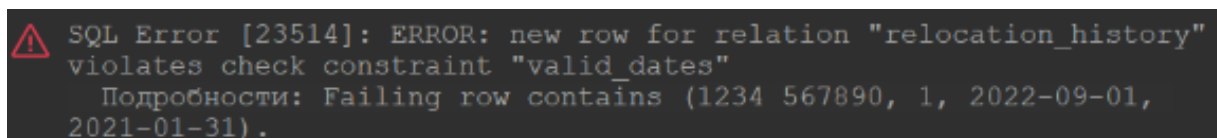


Рис. 24 Проверка валидности дат

Также в разработанной системе есть триггер для проверки, что аспирант и студент не могут являться соседями по комнате и в комнате могут проживать не более 3 человек.

Проверим можно ли добавить в комнату 4-го человека.

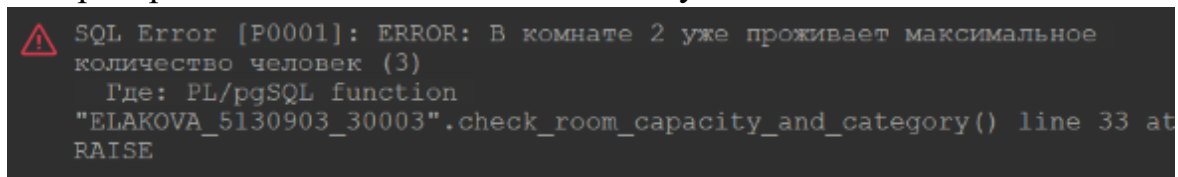


Рис. 25 Проверка на количество проживающих в одной комнате

Теперь требуется проверить, что в комнате не могут жить аспирант и студент. На рис. 26 изображен успешный результат проверки.

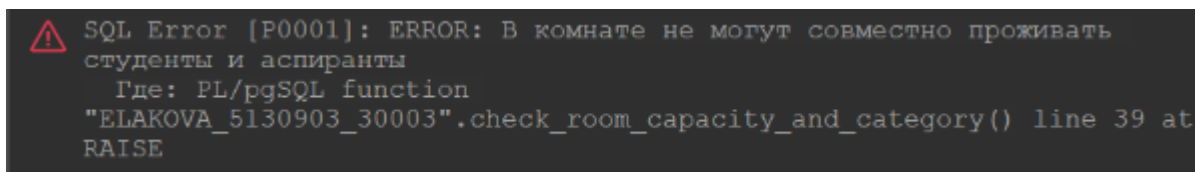


Рис. 26 Проверка на возможность сожительства аспиранта и студента

В системе еще прописан триггер для проверки, что молодой преподаватель живет в одиночестве. Сначала заселим одного преподавателя в комнату 4, затем попытаемся подселить к нему другого человека. На рис. 27 можно увидеть, что триггер исправно работает.

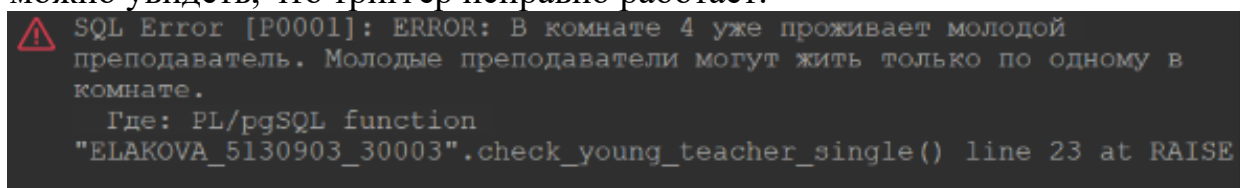


Рис. 27 Проверка триггера для молодых преподавателей

Обратное проектирование

Проведем обратное проектирование: выведем диаграмму (см. рис. 20 Физическая модель).

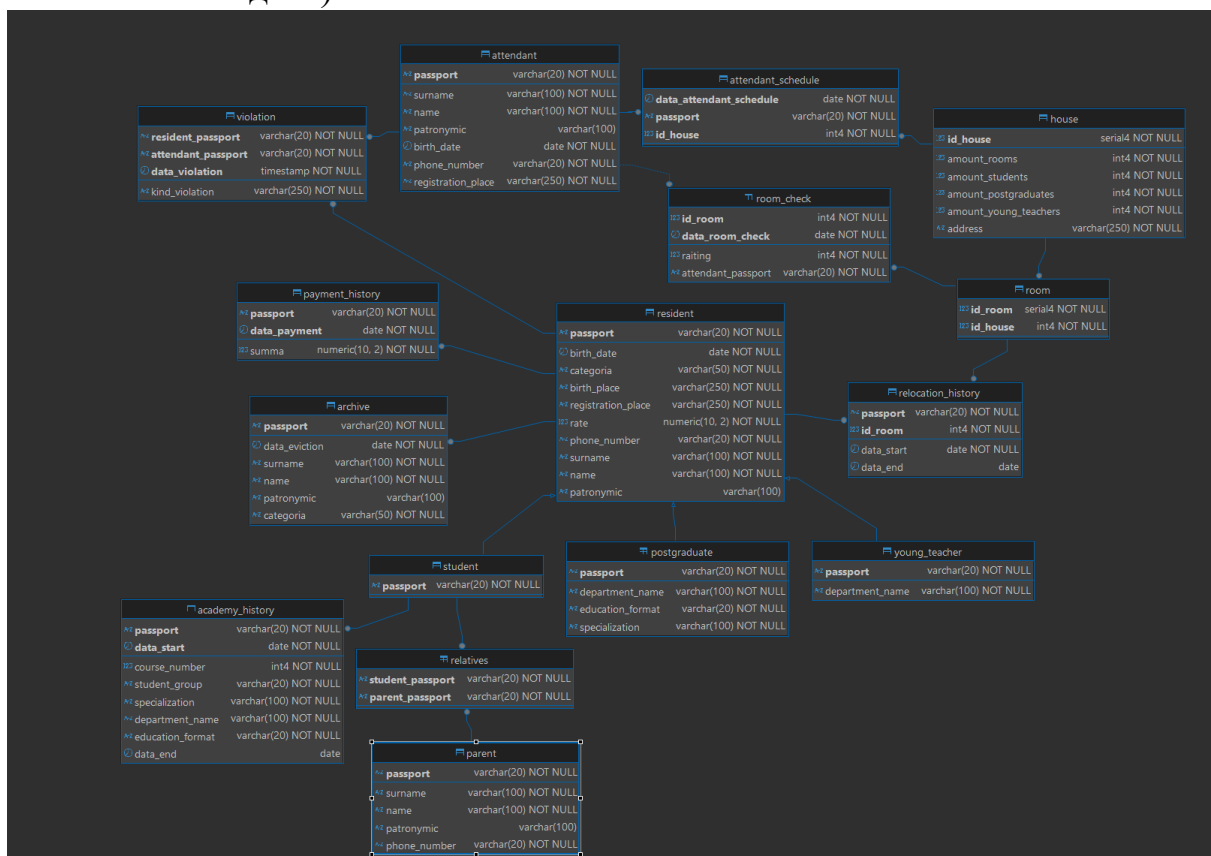


Рис. 28 Физическая модель

РАЗРАБОТКА ЗАПРОСОВ

Для проверки работоспособности получившейся информационной системы необходимо составить SELECT-запросы.

Найдем общую сумму платежей по каждому жильцу. На рис. 29 представлены все данные, которые есть в БД.

Таблица	passport	data_payment	summa
1	2345 678901	2023-02-15 GMT+03:00	5 000
2	1234 567890	2023-01-20 GMT+03:00	5 000
3	3456 789012	2023-01-10 GMT+03:00	7 000
4	4567 890123	2023-01-05 GMT+03:00	10 000
5	2345 678901	2023-03-15 GMT+03:00	5 000
6	4567 890123	2023-02-05 GMT+03:00	10 000

Рис. 29 Все данные из payment_history

SQL- запрос будет выглядеть следующим образом:

SELECT

r.surname || ' ' || r.name AS resident_name,

r.categoria,

SUM(ph.summa) AS total_payments,

COUNT(ph.passport) AS payments_count

FROM "ELAKOVA_5130903_30003".Payment_history ph

JOIN "ELAKOVA_5130903_30003".Resident r ON ph.passport = r.passport

GROUP BY resident_name, r.categoria

HAVING SUM(ph.summa) > 0

ORDER BY total_payments DESC;

Результат, который был подсчитан (см. рис. 30):

Таблица	resident_name	categoria	total_payments	payments_count
1	Васильева Ольга	молодой преподаватель	20 000	2
2	Кузнецова Елена	студент	10 000	2
3	Попов Дмитрий	аспирант	7 000	1
4	Смирнов Алексей	студент	5 000	1

Рис. 30 Результат запроса

Далее найдем средний рейтинг комнат по общежитиям. Данные, которые хранятся в таблице Room_check представлены на рис. 31.

	id_room	data_room_check	rating	attendant_passport
1	1	2023-05-01 GMT+03:00	95	4510 123456
2	2	2023-05-01 GMT+03:00	85	4510 123456
3	6	2023-05-02 GMT+03:00	90	4510 654321
4	7	2023-05-02 GMT+03:00	80	4510 654321
5	11	2023-05-03 GMT+03:00	92	4510 987654
6	12	2023-05-03 GMT+03:00	88	4510 987654

Рис. 31 Все данные таблицы room_check

SQL-запрос выглядит следующим образом:

```
SELECT h.address, ROUND(AVG(rc.rating), 1) AS avg_room_rating,
       COUNT(rc.ID_room) AS checks_count
FROM "ELAKOVA_5130903_30003".Room_check rc
JOIN "ELAKOVA_5130903_30003".Room r ON rc.ID_room = r.ID_room
JOIN "ELAKOVA_5130903_30003".House h ON r.ID_house = h.ID_house
GROUP BY h.address
ORDER BY avg_room_rating DESC;
```

Результат запроса представлен на рис. 32.

	address	avg_room_rating	checks_count
1	ул. Молодежная, д.10	90	2
2	ул. Студенческая, д.1	90	2
3	ул. Академическая, д.5	85	2

Рис. 32 Результат запроса

Необходимо найти количество жильцов по комнатам с указанием категории проживающих. С помощью следующего запроса можно выполнить поставленную задачу.

```
SELECT h.address, rm.ID_room, COUNT(rh.passport) AS residents_count,
       STRING_AGG(DISTINCT res.categoria, ', ') AS resident_types
FROM "ELAKOVA_5130903_30003".Relocation_history rh
JOIN "ELAKOVA_5130903_30003".Room rm ON rh.ID_room = rm.ID_room
JOIN "ELAKOVA_5130903_30003".House h ON rm.ID_house = h.ID_house
```

```
JOIN "ELAKOVA_5130903_30003".Resident res ON rh.passport =
res.passport
```

```
WHERE rh.data_end IS NULL OR rh.data_end > CURRENT_DATE
```

```
GROUP BY h.address, rm.ID_room
```

```
HAVING COUNT(rh.passport) > 0
```

```
ORDER BY h.address, rm.ID_room;
```

Полученный результат представлен на рис. 33.

	address	id_room	residents_count	resident_types
1	ул. Академическая, д.5	6	1	студент
2	ул. Академическая, д.5	7	1	молодой преподаватель
3	ул. Молодежная, д.10	11	1	аспирант
4	ул. Студенческая, д.1	2	3	студент
5	ул. Студенческая, д.1	3	1	студент
6	ул. Студенческая, д.1	4	1	молодой преподаватель

Рис. 33 Результат запроса

Далее написан запрос по поиску дежурных с количеством, проверенных ими комнатами. С помощью данного запроса:

```
SELECT a.surname || ' ' || a.name AS attendant_name,
```

```
COUNT(rc.attendant_passport) AS room_checks_count,
```

```
ROUND(AVG(rc.raiting), 1) AS avg_rating_given
```

```
FROM "ELAKOVA_5130903_30003".Room_check rc
```

```
JOIN "ELAKOVA_5130903_30003".Attendant a ON rc.attendant_passport =
a.passport
```

```
GROUP BY attendant_name
```

```
ORDER BY room_checks_count DESC;
```

Полученный результат представлен на рис. 34.

	attendant_name	room_checks_count	avg_rating_given
1	Иванов Иван	2	90
2	Сидоров Алексей	2	90
3	Петрова Мария	2	85

Рис. 34 Результат запроса

Последним запросом к БД будет поиск среднего возраста жильцов по категориям.

```
SELECT
CASE
    WHEN categoria = 'студент' THEN 'Студенты'
    WHEN categoria = 'аспирант' THEN 'Аспиранты'
    ELSE 'Молодые преподаватели'
END AS category,
ROUND(AVG(EXTRACT(YEAR FROM AGE(CURRENT_DATE,
birth_date)))) AS avg_age,
COUNT(*) AS count
FROM "ELAKOVA_5130903_30003".Resident
JOIN "ELAKOVA_5130903_30003".Relocation_history rh
    ON Resident.passport = rh.passport
WHERE rh.data_end IS NULL OR rh.data_end > CURRENT_DATE
GROUP BY category
ORDER BY avg_age;
```

Полученный результат представлен на рис. 35.

	category	avg_age	count
1	Студенты	24	5
2	Аспиранты	29	1
3	Молодые преподаватели	34	2

Рис. 35 Результат запроса

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель – спроектирована и реализована реляционная база данных для эффективного управления информацией об общежитии. Автоматизация процессов учета проживающих, оплат и иных ключевых аспектов позволяет минимизировать ошибки, сократить временные затраты и оптимизировать использование ресурсов.

Для достижения поставленной цели были выполнены следующие задачи:

- Проведен анализ предметной области, определены бизнес-правила и выявлены основные сущности с их взаимосвязями.
- Разработаны концептуальная, логическая и физические модели базы данных.
- Проведена нормализация базы данных, что обеспечило отсутствие избыточности и аномалий данных.
- Создано представление базы данных, соответствующее поставленным требованиям.
- Протестирована корректность работы базы данных, подтверждена ее функциональность и соответствие требованиям.

Разработанная система предоставляет удобный инструмент для администрирования общежития, что является обоснованием актуальности выбранной темы. В перспективе в базе данных можно усовершенствовать безопасность личных данных, добавить шифрование данных.

СПИСОК ЛИТЕРАТУРЫ

1. Официальная документация PostgreSQL [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/> (дата обращения 01.05.2025).
2. Официальная документация DBeaver [Электронный ресурс]. – URL: <https://dbeaver.com/docs/dbeaver/> (дата обращения 01.05.2025).
3. Правила внутреннего распорядка в общежитиях студенческого городка [Электронный ресурс]. – URL: https://www.spbstu.ru/upload/campus/hostel_intern_regulation.pdf (дата обращения 25.04.2025).
4. Положение о порядке оплаты за проживание в студенческих общежитиях, секциях, блоках улучшенного проживания [Электронный ресурс]. – URL: https://www.spbstu.ru/upload/campus/Prilozhenie_9._Polozhenie_o_poryadke_oplaty_za_prozhivanie_v_studencheskih_obschezhitiyah_sektsiyah_bloках_uluchshennogo_prozhivaniya.pdf (дата обращения 25.04.2025).
5. Карпова Т. С. Базы данных: модели, разработка, реализация. – СПб.: Питер, 2019.
6. Коннолли Т., Бегг К. Базы данных: проектирование, реализация и сопровождение. – М.: Вильямс, 2018.
7. Дейт К. Дж. Введение в системы баз данных. – М.: Вильямс, 2020.
8. Кузнецов С.Г. Основы баз данных. – М.: Национальный Открытый Университет ИНТУИТ, 2024.
9. Моргунов Е. П. PostgreSQL. Основы языка SQL. – СПб.: «БХВ-Петербург», 2018.
10. Комаров В. И. Путеводитель по базам данных. — М.: ДМК-Пресс, 2024.

ПРИЛОЖЕНИЕ

Приложение 1

Код для создания БД:

```
CREATE TABLE "ELAKOVA_5130903_30003".Resident (
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
    birth_date DATE NOT NULL,
    categoria VARCHAR(50) NOT NULL CHECK (
        categoria IN ('студент', 'аспирант', 'молодой преподаватель')),
    birth_place VARCHAR(250) NOT NULL,
    registration_place VARCHAR(250) NOT NULL,
    rate DECIMAL(10, 2) NOT NULL,
    phone_number VARCHAR(20) NOT NULL CHECK (phone_number ~
        '^\\+?[0-9]{10,12}$'),
    surname VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    CONSTRAINT age_check CHECK (
        -- Для молодых преподавателей проверяем, что возраст не больше 35
        лет
        (categoria <> 'молодой преподаватель') OR
        (EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date)) <= 35)
    )
);

CREATE TABLE "ELAKOVA_5130903_30003".Student (
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$')
) INHERITS ("ELAKOVA_5130903_30003".Resident);
```

```

CREATE TABLE "ELAKOVA_5130903_30003".Postgraduate(
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
    department_name VARCHAR(100) NOT NULL,
    education_format VARCHAR(20) NOT NULL
CHECK(education_format IN ('бюджет', 'контракт')),
    specialization VARCHAR(100) NOT NULL
) INHERITS ("ELAKOVA_5130903_30003".Resident);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Young_teacher (
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
    department_name VARCHAR(100) NOT NULL
) INHERITS ("ELAKOVA_5130903_30003".Resident);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Parent (
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
    surname VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    phone_number VARCHAR(20) NOT NULL CHECK (phone_number ~
'^\+?[0-9]{10,12}$')
);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Relatives(
    student_passport VARCHAR(20) NOT NULL CHECK(student_passport
~ '^[0-9]{4}[ ]?[0-9]{6}$'),

```

```

        parent_passport VARCHAR(20) NOT NULL CHECK(parent_passport ~
'^[0-9]{4}[ ]?[0-9]{6}$'),
        PRIMARY KEY(student_passport, parent_passport)
    );

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Relatives
ADD CONSTRAINT fk_student
FOREIGN KEY (student_passport)
REFERENCES "ELAKOVA_5130903_30003".Student(passport);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Relatives
ADD CONSTRAINT fk_parent
FOREIGN KEY (parent_passport)
REFERENCES "ELAKOVA_5130903_30003".Parent(passport);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Academy_history(
    passport VARCHAR(20) NOT NULL CHECK(passport ~ '[0-9]{4}[
]?[0-9]{6}$'),
    course_number INTEGER NOT NULL CHECK (course_number
BETWEEN 1 AND 6),
    student_group VARCHAR(20) NOT NULL CHECK(student_group ~
'^[0-9]{7}/[0-9]{5}$'),
    specialization VARCHAR(100) NOT NULL,
    department_name VARCHAR(100) NOT NULL,
    education_format          VARCHAR(20)          NOT          NULL
CHECK(education_format IN ('бюджет', 'контракт')),
    data_start DATE NOT NULL,
    data_end DATE,
    PRIMARY KEY (passport, data_start),

```



```

        CONSTRAINT valid_study_dates CHECK (data_end IS NULL OR
data_end > data_start)
    );

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Academy_history
ADD CONSTRAINT fk_academy_history
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Student(passport);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Payment_history (
    passport VARCHAR(20) NOT NULL CHECK(passport ~ '^[0-9]{4}[
]?[0-9]{6}$'),
    data_payment DATE NOT NULL,
    summa DECIMAL(10, 2) NOT NULL CHECK (summa > 0),
    PRIMARY KEY(passport, data_payment)
);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Payment_history
ADD CONSTRAINT fk_payment_history
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Archive (
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-
9]{4}[ ]?[0-9]{6}$'),
    data_eviction DATE NOT NULL,
    surname VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),

```

```

        categoria VARCHAR(50) NOT NULL CHECK (
        categoria IN ('студент', 'аспирант', 'молодой преподаватель'))
    );

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Archive
ADD CONSTRAINT fk_archive
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".House (
    ID_house SERIAL PRIMARY KEY,
    amount_rooms INTEGER NOT NULL CHECK(amount_rooms>0),
    amount_students INTEGER NOT NULL CHECK(amount_students>=0)
,
    amount_postgraduates          INTEGER          NOT NULL
CHECK(amount_postgraduates>=0) ,
    amount_young_teachers          INTEGER          NOT NULL
CHECK(amount_young_teachers>=0),
    address VARCHAR(250) NOT NULL
);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Room (
    ID_room SERIAL PRIMARY KEY,
    ID_house INTEGER NOT NULL CHECK(ID_house>0)
);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Room
DROP CONSTRAINT room_pkey;

```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room
ADD PRIMARY KEY (ID_room, ID_house);
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room
ADD CONSTRAINT fk_room
FOREIGN KEY (ID_house)
REFERENCES "ELAKOVA_5130903_30003".House(ID_house);
```

```
CREATE TABLE "ELAKOVA_5130903_30003".Attendant(
    passport VARCHAR(20) PRIMARY KEY CHECK(passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
    surname VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    birth_date DATE NOT NULL,
    phone_number VARCHAR(20) NOT NULL CHECK (phone_number ~
'^\+?[0-9]{10,12}$'),
    registration_place VARCHAR(250) NOT NULL
);
```

```
CREATE TABLE "ELAKOVA_5130903_30003".Attendant_schedule(
    data_attendant_schedule DATE NOT NULL,
    passport VARCHAR(20) NOT NULL REFERENCES
Attendant(passport),
    ID_house INTEGER NOT NULL REFERENCES House(ID_house),
    PRIMARY KEY(data_attendant_schedule,passport,ID_house)
);
```

```
CREATE TABLE "ELAKOVA_5130903_30003".Room_check (
    ID_room INTEGER NOT NULL,
```

```

        data_room_check DATE NOT NULL,
        raiting INTEGER NOT NULL CHECK(raiting BETWEEN 0 AND 100),
        attendant_passport          VARCHAR(20)          NOT          NULL
CHECK(attendant_passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
        PRIMARY KEY(ID_room,data_room_check)
);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Room
ADD CONSTRAINT uk_room_id UNIQUE (ID_room);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Room_check
ADD CONSTRAINT fk_room_check_room
FOREIGN KEY (ID_room)
REFERENCES "ELAKOVA_5130903_30003".Room(ID_room);

```

```

CREATE TABLE "ELAKOVA_5130903_30003".Violation(
        resident_passport          VARCHAR(20)          NOT          NULL
CHECK(resident_passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
        attendant_passport          VARCHAR(20)          NOT          NULL
CHECK(resident_passport ~ '^[0-9]{4}[ ]?[0-9]{6}$'),
        data_violation TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        kind_violation VARCHAR(250) NOT NULL,
        PRIMARY KEY(resident_passport,attendant_passport,data_violation)
);

```

```

ALTER TABLE "ELAKOVA_5130903_30003".Violation
ADD CONSTRAINT fk_attendant_violation
FOREIGN KEY (attendant_passport)
REFERENCES "ELAKOVA_5130903_30003".Attendant(passport);

```

```
ALTER TABLE "ELAKOVA_5130903_30003".Violation
ADD CONSTRAINT fk_resident_violation
FOREIGN KEY (resident_passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport);
```

```
CREATE TABLE "ELAKOVA_5130903_30003".Relocation_history (
    passport VARCHAR(20) NOT NULL CHECK(passport ~ '^[0-9]{4}[
]?[0-9]{6}$') REFERENCES Resident(passport),
    ID_room INTEGER NOT NULL REFERENCES Room(ID_room),
    data_start DATE NOT NULL,
    data_end DATE,
    PRIMARY KEY(passport, ID_room),
    CONSTRAINT valid_dates CHECK (data_end IS NULL OR data_end >
data_start)
);
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room_check
ADD CONSTRAINT fk_room_check_attendant
FOREIGN KEY (attendant_passport)
REFERENCES "ELAKOVA_5130903_30003".Attendant(passport);
```

```
-- Удаляем старые ограничения внешних ключей (если они существуют)
ALTER TABLE "ELAKOVA_5130903_30003".Relatives
DROP CONSTRAINT IF EXISTS fk_student,
DROP CONSTRAINT IF EXISTS fk_parent;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Academy_history
DROP CONSTRAINT IF EXISTS fk_academy_history;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Payment_history
```

```
DROP CONSTRAINT IF EXISTS fk_payment_history;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Archive  
DROP CONSTRAINT IF EXISTS fk_archive;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room  
DROP CONSTRAINT IF EXISTS fk_room;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Attendant_schedule  
DROP CONSTRAINT IF EXISTS attendant_schedule_passport_fkey,  
DROP CONSTRAINT IF EXISTS attendant_schedule_id_house_fkey;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room_check  
DROP CONSTRAINT IF EXISTS fk_room_check_room,  
DROP CONSTRAINT IF EXISTS fk_room_check_attendant;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Violation  
DROP CONSTRAINT IF EXISTS fk_attendant_violation,  
DROP CONSTRAINT IF EXISTS fk_resident_violation;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Relocation_history  
DROP CONSTRAINT IF EXISTS relocation_history_passport_fkey,  
DROP CONSTRAINT IF EXISTS relocation_history_id_room_fkey;
```

```
-- Добавляем новые ограничения с каскадными действиями
```

```
-- Для таблицы Relatives
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Relatives  
ADD CONSTRAINT fk_student  
FOREIGN KEY (student_passport)
```

```
REFERENCES "ELAKOVA_5130903_30003".Student(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Relatives
ADD CONSTRAINT fk_parent
FOREIGN KEY (parent_passport)
REFERENCES "ELAKOVA_5130903_30003".Parent(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
-- Для таблицы Academy_history
ALTER TABLE "ELAKOVA_5130903_30003".Academy_history
ADD CONSTRAINT fk_academy_history
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Student(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
-- Для таблицы Payment_history
ALTER TABLE "ELAKOVA_5130903_30003".Payment_history
ADD CONSTRAINT fk_payment_history
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
-- Для таблицы Archive
ALTER TABLE "ELAKOVA_5130903_30003".Archive
ADD CONSTRAINT fk_archive
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

-- Для таблицы Room

```
ALTER TABLE "ELAKOVA_5130903_30003".Room  
ADD CONSTRAINT fk_room  
FOREIGN KEY (ID_house)  
REFERENCES "ELAKOVA_5130903_30003".House(ID_house)  
ON DELETE CASCADE ON UPDATE CASCADE;
```

-- Для таблицы Attendant_schedule

```
ALTER TABLE "ELAKOVA_5130903_30003".Attendant_schedule  
ADD CONSTRAINT attendant_schedule_passport_fkey  
FOREIGN KEY (passport)  
REFERENCES "ELAKOVA_5130903_30003".Attendant(passport)  
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Attendant_schedule  
ADD CONSTRAINT attendant_schedule_id_house_fkey  
FOREIGN KEY (ID_house)  
REFERENCES "ELAKOVA_5130903_30003".House(ID_house)  
ON DELETE CASCADE ON UPDATE CASCADE;
```

-- Для таблицы Room_check

```
ALTER TABLE "ELAKOVA_5130903_30003".Room_check  
ADD CONSTRAINT fk_room_check_room  
FOREIGN KEY (ID_room)  
REFERENCES "ELAKOVA_5130903_30003".Room(ID_room)  
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Room_check  
ADD CONSTRAINT fk_room_check_attendant  
FOREIGN KEY (attendant_passport)
```



```
REFERENCES "ELAKOVA_5130903_30003".Attendant(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
-- Для таблицы Violation
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Violation
ADD CONSTRAINT fk_attendant_violation
FOREIGN KEY (attendant_passport)
REFERENCES "ELAKOVA_5130903_30003".Attendant(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Violation
ADD CONSTRAINT fk_resident_violation
FOREIGN KEY (resident_passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
-- Для таблицы Relocation_history
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Relocation_history
ADD CONSTRAINT relocation_history_passport_fkey
FOREIGN KEY (passport)
REFERENCES "ELAKOVA_5130903_30003".Resident(passport)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "ELAKOVA_5130903_30003".Relocation_history
ADD CONSTRAINT relocation_history_id_room_fkey
FOREIGN KEY (ID_room)
REFERENCES "ELAKOVA_5130903_30003".Room(ID_room)
ON DELETE CASCADE ON UPDATE CASCADE;
```

```
CREATE OR REPLACE FUNCTION
"ELAKOVA_5130903_30003".check_room_capacity_and_category()
```

RETURNS TRIGGER AS \$\$

DECLARE

current_residents INTEGER := 0;

has_student BOOLEAN := FALSE;

has_postgraduate BOOLEAN := FALSE;

resident_category VARCHAR(50);

new_resident_category VARCHAR(50);

BEGIN

-- Получаем категорию нового жильца

SELECT categoria INTO new_resident_category

FROM "ELAKOVA_5130903_30003".Resident

WHERE passport = NEW.passport;

-- Проверяем текущих жильцов в комнате (у которых data_end IS NULL
или в будущем)

FOR resident_category IN

SELECT r.categoria

FROM "ELAKOVA_5130903_30003".Relocation_history rh

JOIN "ELAKOVA_5130903_30003".Resident r ON rh.passport =

r.passport

WHERE rh.ID_room = NEW.ID_room

AND (rh.data_end IS NULL OR rh.data_end > CURRENT_DATE)

LOOP

current_residents := current_residents + 1;

IF resident_category = 'студент' THEN

has_student := TRUE;

ELSIF resident_category = 'аспирант' THEN

has_postgraduate := TRUE;

END IF;

```
END LOOP;
```

```
-- Проверка на максимальное количество жильцов (должно быть >= 3, а не > 3)
```

```
IF (NEW.data_end IS NULL OR NEW.data_end > CURRENT_DATE)
AND current_residents >= 3 THEN
```

```
    RAISE EXCEPTION 'В комнате % уже проживает максимальное
количество человек (3)', NEW.ID_room;
```

```
END IF;
```

```
-- Проверка на смешанное проживание студентов и аспирантов
```

```
IF (new_resident_category = 'студент' AND has_postgraduate) OR
    (new_resident_category = 'аспирант' AND has_student) THEN
```

```
    RAISE EXCEPTION 'В комнате не могут совместно проживать
студенты и аспиранты';
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_room_capacity_and_category_insert
```

```
BEFORE INSERT ON "ELAKOVA_5130903_30003".Relocation_history
```

```
FOR EACH ROW
```

```
EXECUTE
```

```
FUNCTION
```

```
"ELAKOVA_5130903_30003".check_room_capacity_and_category();
```

```
CREATE TRIGGER trg_check_room_capacity_and_category_update
```

```
BEFORE UPDATE ON "ELAKOVA_5130903_30003".Relocation_history
```

```
FOR EACH ROW
```

```

EXECUTE FUNCTION
"ELAKOVA_5130903_30003".check_room_capacity_and_category();

```

```

CREATE OR REPLACE FUNCTION
"ELAKOVA_5130903_30003".check_young_teacher_single()
RETURNS TRIGGER AS $$
DECLARE
    resident_count INTEGER;
    is_young_teacher BOOLEAN;
BEGIN
    -- Проверяем, является ли новый жилец молодым преподавателем
    SELECT (categoria = 'молодой преподаватель') INTO is_young_teacher
    FROM "ELAKOVA_5130903_30003".Resident
    WHERE passport = NEW.passport;

    -- Если это молодой преподаватель
    IF is_young_teacher THEN
        -- Проверяем, есть ли уже молодые преподаватели в этой комнате
        SELECT COUNT(*) INTO resident_count
        FROM "ELAKOVA_5130903_30003".Relocation_history rh
        JOIN "ELAKOVA_5130903_30003".Resident r ON rh.passport =
r.passport
        WHERE rh.ID_room = NEW.ID_room
        AND (rh.data_end IS NULL OR rh.data_end > CURRENT_DATE)
        AND r.categoria = 'молодой преподаватель';

        -- Если уже есть молодой преподаватель в комнате
        IF resident_count > 0 THEN

```

```

        RAISE EXCEPTION 'В комнате % уже проживает молодой
преподаватель. Молодые преподаватели могут жить только по одному в
комнате.', NEW.ID_room;

```

```

        END IF;

```

```

    END IF;

```

```

    -- Проверяем, не пытаются ли подселить кого-то к молодому
преподавателю

```

```

    IF NOT is_young_teacher THEN

```

```

        -- Проверяем, есть ли в комнате молодой преподаватель

```

```

        SELECT COUNT(*) INTO resident_count

```

```

        FROM "ELAKOVA_5130903_30003".Relocation_history rh

```

```

        JOIN "ELAKOVA_5130903_30003".Resident r ON rh.passport =
r.passport

```

```

        WHERE rh.ID_room = NEW.ID_room

```

```

        AND (rh.data_end IS NULL OR rh.data_end > CURRENT_DATE)

```

```

        AND r.categoria = 'молодой преподаватель';

```

```

    -- Если в комнате есть молодой преподаватель

```

```

    IF resident_count > 0 THEN

```

```

        RAISE EXCEPTION 'В комнате % проживает молодой
преподаватель. К молодым преподавателям нельзя подселять других жильцов.',
NEW.ID_room;

```

```

        END IF;

```

```

    END IF;

```

```

    RETURN NEW;

```

```

END;

```

```

$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_check_young_teacher_single_insert
BEFORE INSERT ON "ELAKOVA_5130903_30003".Relocation_history
FOR EACH ROW
EXECUTE FUNCTION
"ELAKOVA_5130903_30003".check_young_teacher_single();

```

```

CREATE TRIGGER trg_check_young_teacher_single_update
BEFORE UPDATE ON "ELAKOVA_5130903_30003".Relocation_history
FOR EACH ROW
EXECUTE FUNCTION
"ELAKOVA_5130903_30003".check_young_teacher_single();

```

--ПРЕДСТАВЛЕНИЕ

```

CREATE OR REPLACE VIEW
"ELAKOVA_5130903_30003".residents_simple_view AS
SELECT
    r.surname || ' ' || r.name || COALESCE(' ' || r.patronymic, '') AS full_name,
    CASE
        WHEN r.categoria = 'студент' THEN 'Студент'
        WHEN r.categoria = 'аспирант' THEN 'Аспирант'
        WHEN r.categoria = 'молодой преподаватель' THEN 'Молодой
преподаватель'
        ELSE r.categoria
    END AS resident_category,
    r.rate AS monthly_rate
FROM
    "ELAKOVA_5130903_30003".Resident r
JOIN

```

```

"ELAKOVA_5130903_30003".Relocation_history rh ON r.passport =
rh.passport
WHERE
    -- Условие для выборки проживавших в учебном году (01.09.2021 -
    05.07.2022)
    (rh.data_start <= '2022-07-05' AND (rh.data_end IS NULL OR rh.data_end
    >= '2021-09-01'))
GROUP BY
    r.surname, r.name, r.patronymic, r.categoria, r.rate
ORDER BY
    r.surname, r.name;

```

Приложение 2

Заполнение тестовыми данными:

```

INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
VALUES
    ('1234 567890', '2000-01-15', 'студент', 'Москва', 'Москва', 5000.00,
    '+79161112233', 'Смирнов', 'Алексей', 'Петрович'),
    ('2345 678901', '2001-03-20', 'студент', 'Санкт-Петербург', 'Санкт-
    Петербург', 5000.00, '+79162223344', 'Кузнецова', 'Елена', 'Игоревна'),
    ('3456 789012', '1995-06-25', 'аспирант', 'Новосибирск', 'Новосибирск',
    7000.00, '+79163334455', 'Попов', 'Дмитрий', 'Сергеевич'),
    ('4567 890123', '1990-08-30', 'молодой преподаватель', 'Екатеринбург',
    'Екатеринбург', 10000.00, '+79164445566', 'Васильева', 'Ольга', 'Александровна'),
    ('5678 901234', '1989-11-10', 'молодой преподаватель', 'Казань', 'Казань',
    10000.00, '+79165556677', 'Николаев', 'Артем', 'Владимирович');

```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES
```

```
('7777 901234', '1991-11-12', 'молодой преподаватель', 'Казань', 'Казань',
10000.00, '+79165556655', 'Никомнкин', 'Андрей', 'Владимирович'),
```

```
('8888 901234', '1991-11-11', 'молодой преподаватель', 'Казань', 'Казань',
10000.00, '+79165446677', 'Николаев', 'Аид', 'Владимирович');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES
```

```
('4444 567890', '2001-10-15', 'студент', 'Москва', 'Москва', 5000.00,
'+79161144233', 'Смирнов', 'Александр', 'Петрович'),
```

```
('3333 567890', '2000-11-22', 'студент', 'Москва', 'Москва', 5000.00,
'+79144112233', 'Петров', 'Алексей', 'Семенович');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES
```

```
('2222 789012', '1997-06-23', 'аспирант', 'Новосибирск', 'Новосибирск',
7000.00, '+79163334455', 'Иванов', 'Илья', 'Сергеевич')
```

```
INSERT INTO "ELAKOVA_5130903_30003".Student (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES
```



```
('1234 567890', '2000-01-15', 'студент', 'Москва', 'Москва', 5000.00,
'+79161112233', 'Смирнов', 'Алексей', 'Петрович'),
```

```
('2345 678901', '2001-03-20', 'студент', 'Санкт-Петербург', 'Санкт-
Петербург', 5000.00, '+79162223344', 'Кузнецова', 'Елена', 'Игоревна');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Postgraduate (passport,
department_name, education_format, specialization, birth_date, categoria, birth_place,
registration_place, rate, phone_number, surname, name, patronymic)
```

```
VALUES
```

```
('3456 789012', 'Кафедра информатики', 'бюджет', 'Компьютерные науки',
'1995-06-25', 'аспирант', 'Новосибирск', 'Новосибирск', 7000.00, '+79163334455',
'Попов', 'Дмитрий', 'Сергеевич');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Young_teacher (passport,
department_name, birth_date, categoria, birth_place, registration_place, rate,
phone_number, surname, name, patronymic)
```

```
VALUES
```

```
('4567 890123', 'Кафедра математики', '1990-08-30', 'молодой
преподаватель', 'Екатеринбург', 'Екатеринбург', 10000.00, '+79164445566',
'Васильева', 'Ольга', 'Александровна'),
```

```
('5678 901234', 'Кафедра физики', '1989-11-10', 'молодой преподаватель',
'Казань', 'Казань', 10000.00, '+79165556677', 'Николаев', 'Артем', 'Владимирович'
);
```

```
-- Заполнение таблицы Parent (родители)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Parent (passport, surname,
name, patronymic, phone_number)
```

```
VALUES
```

```
('1111 222233', 'Смирнов', 'Петр', 'Алексеевич', '+79167778899'),
```

```
('2222 333344', 'Кузнецова', 'Ирина', 'Михайловна', '+79168889900'),
```

```
('3333 444455', 'Попова', 'Светлана', 'Дмитриевна', '+79169990011');
```

```
-- Заполнение таблицы Relatives (родственники)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Relatives (student_passport,  
parent_passport)
```

```
VALUES
```

```
('1234 567890', '1111 222233'),
```

```
('2345 678901', '2222 333344'),
```

```
('1234 567890', '3333 444455');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Academy_history (passport,  
course_number, student_group, specialization, department_name, education_format,  
data_start, data_end)
```

```
VALUES
```

```
('1234 567890', 2, '1234567/54321', 'Информатика', 'Факультет  
компьютерных наук', 'бюджет', '2022-09-01', NULL),
```

```
('2345 678901', 1, '2345678/65432', 'Математика', 'Факультет математики',  
'контракт', '2023-09-01', NULL);
```

```
-- Заполнение таблицы Attendant (дежурные)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Attendant (passport, surname,  
name, patronymic, birth_date, phone_number, registration_place)
```

```
VALUES
```

```
('4510 123456', 'Иванов', 'Иван', 'Иванович', '1975-05-15', '+79161234567', 'г.  
Москва, ул. Ленина, д.1'),
```

```
('4510 654321', 'Петрова', 'Мария', 'Сергеевна', '1980-08-20', '+79167654321',  
'г. Москва, ул. Пушкина, д.10'),
```

```
('4510 987654', 'Сидоров', 'Алексей', 'Николаевич', '1978-03-10',  
'+79169876543', 'г. Москва, ул. Гагарина, д.5');
```

```
INSERT INTO "ELAKOVA_5130903_30003".House (amount_rooms,
amount_students, amount_postgraduates, amount_young_teachers, address)
```

```
VALUES
```

```
(100, 300, 50, 20, 'ул. Студенческая, д.1'),
```

```
(80, 240, 40, 15, 'ул. Академическая, д.5'),
```

```
(120, 360, 60, 25, 'ул. Молодежная, д.10');
```

```
-- Заполнение таблицы Room (комнаты)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Room (ID_house)
```

```
VALUES
```

```
(1), (1), (1), (1), (1),
```

```
(2), (2), (2), (2), (2),
```

```
(3), (3), (3), (3), (3);
```

```
INSERT INTO "ELAKOVA_5130903_30003".Attendant_schedule
(data_attendant_schedule, passport, ID_house)
```

```
VALUES
```

```
('2023-05-01', '4510 123456', 1),
```

```
('2023-05-02', '4510 654321', 2),
```

```
('2023-05-03', '4510 987654', 3);
```

```
INSERT INTO "ELAKOVA_5130903_30003".Room_check (ID_room,
data_room_check, rating, attendant_passport)
```

```
VALUES
```

```
(1, '2023-05-01', 95, '4510 123456'),
```

```
(2, '2023-05-01', 85, '4510 123456'),
```

```
(6, '2023-05-02', 90, '4510 654321'),
```

```
(7, '2023-05-02', 80, '4510 654321'),
```

```
(11, '2023-05-03', 92, '4510 987654'),
```

```
(12, '2023-05-03', 88, '4510 987654');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Violation (resident_passport,
attendant_passport, kind_violation)
```

```
VALUES
```

```
('1234 567890', '4510 123456', 'Нарушение тишины после 23:00'),
```

```
('2345 678901', '4510 654321', 'Курение в неподобающем месте');
```

```
-- Заполнение таблицы Relocation_history (история переселений)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Relocation_history (passport,
ID_room, data_start, data_end)
```

```
VALUES
```

```
('1234 567890', 1, '2022-09-01', '2023-01-31'),
```

```
('1234 567890', 3, '2023-02-01', NULL),
```

```
('2345 678901', 6, '2023-09-01', NULL),
```

```
('3456 789012', 11, '2020-09-01', NULL),
```

```
('5678 901234', 7, '2022-09-01', NULL);
```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident(passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES ('6789 012345', '1995-06-24', 'студент', 'Уфа', 'Уфа', 5000.00,
'+79165556698', 'Соколов', 'Андрей', 'Викторович'),
```

```
('7890 123456', '1993-04-02', 'аспирант', 'Уфа', 'Уфа', 7000.00, '+79822461589',
'Морозова', 'Татьяна', 'Александровна')
```

```
-- Заполнение таблицы Archive (архив)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Archive (passport,
data_eviction, surname, name, patronymic, categoria)
```

```
VALUES
```

```
('6789 012345', '2022-06-30', 'Соколов', 'Андрей', 'Викторович', 'студент'),
```

```
('7890 123456', '2022-06-30', 'Морозова', 'Татьяна', 'Александровна',
'аспирант');
```

```
-- Заполнение таблицы Payment_history (история платежей)
```

```
INSERT INTO "ELAKOVA_5130903_30003".Payment_history (passport,
data_payment, summa)
```

```
VALUES
```

```
('2345 678901', '2023-02-15', 5000.00),
```

```
('1234 567890', '2023-01-20', 5000.00),
```

```
('3456 789012', '2023-01-10', 7000.00),
```

```
('4567 890123', '2023-01-05', 10000.00);
```

Приложение 3

Тестовые данные для проверки работоспособности системы:

```
-- ТЕСТОВЫЕ ДАННЫЕ--
```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

```
VALUES
```

```
('5678 901234', '1983-01-01', 'молодой преподаватель', 'Казань', 'Казань',
10000.00, '+79165556642', 'Николаева', 'Алия', 'Владимировна');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Payment_history (passport,
data_payment, summa)
```

```
VALUES
```

```
('2345 678901', '2023-03-15', -7000.00);
```

```
INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date,
categoria, birth_place, registration_place, rate, phone_number, surname, name,
patronymic)
```

VALUES

('5678 901234', '1990-01-01', 'магистрант', 'Казань', 'Казань', 10000.00, '+79165556642', 'Николаева', 'Алия', 'Владимировна');

INSERT INTO "ELAKOVA_5130903_30003".Resident (passport, birth_date, categoria, birth_place, registration_place, rate, phone_number, surname, name, patronymic)

VALUES

('5678 901234', '1990-01-01', 'молодой преподаватель', 'Казань', 'Казань', 10000.00, '911', 'Николаева', 'Алия', 'Владимировна');

INSERT INTO "ELAKOVA_5130903_30003".Relocation_history (passport, ID_room, data_start, data_end)

VALUES

('1234 567890', 1, '2022-09-01', '2021-01-31');

--проверка триггера

INSERT INTO "ELAKOVA_5130903_30003".Relocation_history
(passport, ID_room, data_start)

VALUES

('4444 567890', 2, '2023-01-01'),

('1234 567890', 2, '2023-01-01'),

('2345 678901', 2, '2023-01-01');

INSERT INTO "ELAKOVA_5130903_30003".Relocation_history
(passport, ID_room, data_start)

VALUES

('3333 567890', 8, '2024-01-01'),

```
('2222 789012',8,'2024-01-01');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Relocation_history
(passport, ID_room, data_start)
VALUES ('7777 901234', 4, '2023-01-01');
```

```
INSERT INTO "ELAKOVA_5130903_30003".Relocation_history
(passport, ID_room, data_start)
VALUES ('8888 901234', 4, '2023-01-01');
```

Приложение 4

Разработанные запросы:

```
SELECT
  r.surname || ' ' || r.name AS resident_name,
  r.categoria,
  SUM(ph.summa) AS total_payments,
  COUNT(ph.passport) AS payments_count
FROM "ELAKOVA_5130903_30003".Payment_history ph
JOIN "ELAKOVA_5130903_30003".Resident r ON ph.passport = r.passport
GROUP BY resident_name, r.categoria
HAVING SUM(ph.summa) > 0
ORDER BY total_payments DESC;
```

```
SELECT
  h.address,
  ROUND(AVG(rc.raiting), 1) AS avg_room_rating,
  COUNT(rc.ID_room) AS checks_count
FROM "ELAKOVA_5130903_30003".Room_check rc
JOIN "ELAKOVA_5130903_30003".Room r ON rc.ID_room = r.ID_room
JOIN "ELAKOVA_5130903_30003".House h ON r.ID_house = h.ID_house
GROUP BY h.address
ORDER BY avg_room_rating DESC;
```

```
SELECT
  h.address,
  rm.ID_room,
  COUNT(rh.passport) AS residents_count,
```

```

    STRING_AGG(DISTINCT res.categoria, ', ') AS resident_types
FROM "ELAKOVA_5130903_30003".Relocation_history rh
JOIN "ELAKOVA_5130903_30003".Room rm ON rh.ID_room = rm.ID_room
JOIN "ELAKOVA_5130903_30003".House h ON rm.ID_house = h.ID_house
JOIN "ELAKOVA_5130903_30003".Resident res ON rh.passport = res.passport
WHERE rh.data_end IS NULL OR rh.data_end > CURRENT_DATE
GROUP BY h.address, rm.ID_room
HAVING COUNT(rh.passport) > 0
ORDER BY h.address, rm.ID_room;

```

```

SELECT a.surname || ' ' || a.name AS attendant_name,
       COUNT(rc.attendant_passport) AS room_checks_count,
       ROUND(AVG(rc.raiting), 1) AS avg_rating_given
FROM "ELAKOVA_5130903_30003".Room_check rc
JOIN "ELAKOVA_5130903_30003".Attendant a ON rc.attendant_passport =
a.passport
GROUP BY attendant_name
ORDER BY room_checks_count DESC;

```

```

SELECT
CASE
    WHEN categoria = 'студент' THEN 'Студенты'
    WHEN categoria = 'аспирант' THEN 'Аспиранты'
    ELSE 'Молодые преподаватели'
END AS category,
ROUND(AVG(EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date))))
AS avg_age,
COUNT(*) AS count
FROM "ELAKOVA_5130903_30003".Resident
JOIN "ELAKOVA_5130903_30003".Relocation_history rh
    ON Resident.passport = rh.passport
WHERE rh.data_end IS NULL OR rh.data_end > CURRENT_DATE
GROUP BY categoria
ORDER BY avg_age;

```